

DATA 221 Homework 6

Chris Low

Problem 1

a)

```
import pandas as pd
import torch

url = "https://vincentarelbundock.github.io/Rdatasets/csv/carData/TitanicSurvival.csv"
df = pd.read_csv(url)

df.head()
df.columns
```

```
Index(['rownames', 'survived', 'sex', 'age', 'passengerClass'], dtype='object')
```

```
# Drop the rownames column and any rows with missing values
df = df.drop(columns=['rownames'])
df = df.dropna()

# Map the 'survived' and 'sex' columns to binary values
df['survived'] = df['survived'].map({'yes': 1, 'no': 0})
df['sex'] = df['sex'].map({'male': 0, 'female': 1})

# Map the 'passengerClass' column to numerical values
df['passengerClass'] = df['passengerClass'].map({
    '1st': 1,
    '2nd': 2,
    '3rd': 3
})
```

```
# Separate our features and target variable
X = df[['age', 'sex', 'passengerClass']]
y = df['survived']

# As mentioned, center the columns of the predictors
X_centered = X - X.mean()

X_tensor = torch.tensor(X_centered.values, dtype=torch.float32)
y_tensor = torch.tensor(y.values, dtype=torch.float32).view(-1, 1)
```

```
print(X_tensor.shape)
print(y_tensor.shape)

print(X_tensor[:5])
print(y_tensor[:5])
```

```
torch.Size([1046, 3])
torch.Size([1046, 1])
tensor([[ -0.8811,  0.6291, -1.2075],
        [-28.9644, -0.3709, -1.2075],
        [-27.8811,  0.6291, -1.2075],
         [ 0.1189, -0.3709, -1.2075],
         [-4.8811,  0.6291, -1.2075]])
tensor([[1.],
        [1.],
        [0.],
        [0.],
        [0.]])
```

b)

```
def sigmoid(z):
    return 1.0 / (1.0 + torch.exp(-z))

def forward_pass(X_in, w):
    z = torch.matmul(X_in, w)
    return sigmoid(z)

# Generate three random weights
```

```

torch.manual_seed(42)
weights = torch.randn(3, 1, dtype=torch.float32)

# (29yo, female=1, 1st class=1)
raw_passenger = torch.tensor([29.0, 1.0, 1.0], dtype=torch.float32)

means_tensor = torch.tensor(X.mean().values, dtype=torch.float32)
centered_passenger = raw_passenger - means_tensor

centered_passenger = centered_passenger.view(1, -1)

probability = forward_pass(centered_passenger, weights)

print(f"Probability of survival (29yo, female, 1st class): {probability.item():.4f}")

```

Probability of survival (29yo, female, 1st class): 0.3778

c) Cross entropy loss and backward pass

```

def cross_entropy_loss(y_pred, y_true):
    eps = 1e-15
    y_pred = torch.clamp(y_pred, eps, 1 - eps)
    return -torch.mean(y_true * torch.log(y_pred) + (1.0 - y_true) * torch.log(1.0 - y_pred))

# Calculate loss given that this passenger survived (y = 1)
test_target = torch.tensor([[1.0]], dtype=torch.float32)
passenger_loss = cross_entropy_loss(probability, test_target)
print(f"Cross Entropy Loss for this passenger: {passenger_loss.item():.4f}")

# --- Part 1c (Gradient): Backward Pass ---
def backward_pass(X_in, y_pred, y_true):
    # Gradient of BCE loss w.r.t weights
    N = X_in.shape[0]
    gradient = torch.matmul(X_in.t(), (y_pred - y_true)) / N
    return gradient

```

Cross Entropy Loss for this passenger: 0.9733

```
def cross_entropy_loss(p, y):
    # Add small epsilon to avoid log(0)
    eps = 1e-8
    loss = -torch.sum(
        y * torch.log(p + eps) +
        (1 - y) * torch.log(1 - p + eps)
    )
    return loss

y_single = torch.tensor([[1.0]])

loss_single = cross_entropy_loss(probability, y_single)

print(f"Cross entropy for that passenger: {loss_single.item():.4f}")
```

Cross entropy for that passenger: 0.9733

```
torch.manual_seed(42)
weights = torch.randn(3, 1, dtype=torch.float32, requires_grad=True)

# Forward pass
prob = forward_pass(centered_passenger, weights)

# Compute loss
loss = cross_entropy_loss(prob, y_single)

# Backward pass
loss.backward()

print("Gradient of loss wrt weights:")
print(weights.grad)
```

Gradient of loss wrt weights:
 tensor([[0.5482],
 [-0.3914],
 [0.7512]])